

EX NO : 1

DATA ENGINEERING

DATE : 12-06-26

AIM:

Understanding and Exploring Different Types of Data

PROBLEM STATEMENT:

You have been provided with three datasets:

1. Student performance data (CSV file) - Contains demographic, social, family, and academic information of students. Used to analyze factors affecting student performance and predict final grades.

Rows - 395 and Columns - 33

<https://www.kaggle.com/datasets/devansodariya/student-performance-data>

2. Movie reviews (Text file) - Contains movie reviews collected from IMDb along with sentiment labels (positive/negative). Used for sentiment analysis and text mining tasks.

Rows - 50000 and Columns - 2

<https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews>

3. Image dataset - A collection of small color images belonging to 10 different classes such as airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. Used for image classification and computer vision tasks.

Images - 60000

<https://www.cs.toronto.edu/~kriz/cifar.html>

Perform exploratory analysis on each dataset and identify its characteristics.

PART A – TABULAR DATA

Using the student performance dataset:

1. Load the dataset.
2. Display the first 10 records.
3. Identify the number of rows and columns.
4. List all attributes.
5. Check for missing values.
6. Compute basic statistics such as mean, median, minimum, and maximum marks.
7. Identify numerical and categorical attributes.
8. Draw a histogram of marks.
9. Find correlation between numerical attributes.
10. Identify possible target variables for prediction.

CODE:

1. Load the dataset

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
df = pd.read_csv(r"D:\SEM_5\ML\datasets\Table_data\student_data.csv")
```

2. Display the first 10 records

```
print("First 10 Records:")
```

```
print(df.head(10))
```

3. Number of rows and columns

```
rows, cols = df.shape
```

```
print("\nNumber of Rows:", rows)
```

```
print("Number of Columns:", cols)
```

4. List all attributes

```
print("\nAttributes (Columns):")
```

```
print(df.columns.tolist())
```

5. Check for missing values

```
print("\nMissing Values:")
```

```
print(df.isnull().sum())
```

Total missing values

```
print("\nTotal Missing Values:", df.isnull().sum().sum())
```

6. Basic Statistics for Marks Column (G3)

```
print("\nBasic Statistics for Marks (G3):")
```

```
print("Mean Marks:", df['G3'].mean())
```

```
print("Median Marks:", df['G3'].median())
```

```
print("Minimum Marks:", df['G3'].min())
```

```
print("Maximum Marks:", df['G3'].max())
```

7. Identify Numerical and Categorical Attributes

```
numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns
```

```
categorical_cols = df.select_dtypes(include=['object']).columns
```

```
print("\nNumerical Attributes:")
```

```
print(list(numerical_cols))
```

```
print("\nCategorical Attributes:")
```

```
print(list(categorical_cols))
```

8. Draw Histogram of Marks

```
if 'G3' in df.columns:
```

```
    plt.figure(figsize=(8,5))
```

```
    plt.hist(df['G3'], bins=10)
```

```
    plt.title("Histogram of Final Grade (G3)")
```

```
    plt.xlabel("G3 Marks")
```

```
    plt.ylabel("Frequency")
```

```
    plt.grid(True)
```

```
    plt.show()
```

```
else:
```

```
    print("\nColumn 'G3' not found. Use the appropriate marks column.")
```

9. Correlation Between Numerical Attributes

```
correlation_matrix = df[numerical_cols].corr()
```

```
print("\nCorrelation Matrix:")
```

```
print(correlation_matrix)
```

10. Possible Target Variables for Prediction

```
print("\nPossible Target Variables:")
```

```
possible_targets = []
```

```
for col in ['G1', 'G2', 'G3']:
```

```
    if col in df.columns:
```

```
        possible_targets.append(col)
```

```
print(possible_targets)
```

OUTPUT:

2.

First 10 Records:

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	...	goout	Dalc	Walc	health	absences	G1	G2	G3
0	GP	F	18	U	GT3	A	4	4	...	4	1	1	3	6	5	6	6
1	GP	F	17	U	GT3	T	1	1	...	3	1	1	3	4	5	5	6
2	GP	F	15	U	LE3	T	1	1	...	2	2	3	3	10	7	8	10
3	GP	F	15	U	GT3	T	4	2	...	2	1	1	5	2	15	14	15
4	GP	F	16	U	GT3	T	3	3	...	2	1	2	5	4	6	10	10
5	GP	M	16	U	LE3	T	4	3	...	2	1	2	5	10	15	15	15
6	GP	M	16	U	LE3	T	2	2	...	4	1	1	3	0	12	12	11
7	GP	F	17	U	GT3	A	4	4	...	4	1	1	1	6	6	5	6
8	GP	M	15	U	LE3	A	3	2	...	2	1	1	1	0	16	18	19
9	GP	M	15	U	GT3	T	3	4	...	1	1	1	5	0	14	15	15

[10 rows x 33 columns]

3.

Number of Rows: 395
Number of Columns: 33

4.

Attributes (Columns):

['school', 'sex', 'age', 'address', 'famsize', 'Pstatus', 'Medu', 'Fedu', 'Mjob', 'Fjob', 'reason', 'guardian', 'traveltime', 'studytime', 'failures', 'schoolsup', 'famsup', 'paid', 'activities', 'nursery', 'higher', 'internet', 'romantic', 'famrel', 'freetime', 'goout', 'Dalc', 'Walc', 'health', 'absences', 'G1', 'G2', 'G3']

5.

Missing Values:

school	0
sex	0
age	0
address	0
famsize	0
Pstatus	0
Medu	0
Fedu	0
Mjob	0
Fjob	0
reason	0
guardian	0
traveltime	0
studytime	0
failures	0
schoolsup	0
famsup	0
paid	0
activities	0
nursery	0
higher	0
internet	0
romantic	0
famrel	0
freetime	0
goout	0
Dalc	0
Walc	0
health	0

```
absences    0
G1          0
G2          0
G3          0
dtype: int64
```

Total Missing Values: 0

6.

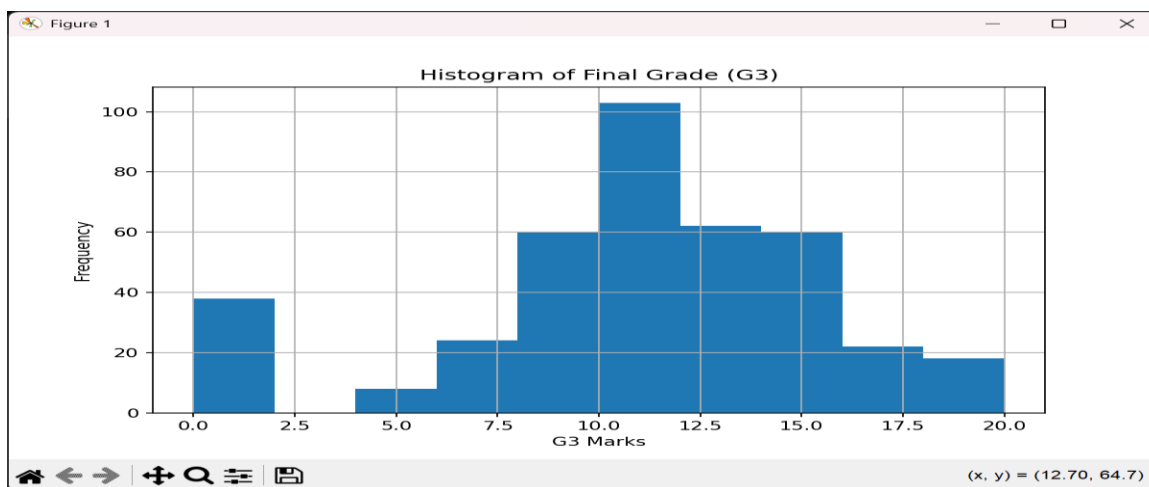
```
Basic Statistics for Marks (G3):
Mean Marks: 10.415189873417722
Median Marks: 11.0
Minimum Marks: 0
Maximum Marks: 20
```

7.

```
Numerical Attributes:
['age', 'Medu', 'Fedu', 'traveltime', 'studytime', 'failures', 'famrel', 'freetime', 'goout', 'Dalc', 'Walc', 'health', 'absences', 'G1', 'G2', 'G3']

Categorical Attributes:
['school', 'sex', 'address', 'famsize', 'Pstatus', 'Mjob', 'Fjob', 'reason', 'guardian', 'schoolsup', 'famsup', 'paid', 'activities', 'nursery', 'higher', 'internet', 'romantic']
```

8.



9.

```
Correlation Matrix:
          age      Medu      Fedu  traveltime  ...  absences      G1      G2      G3
age      1.000000 -0.163658 -0.163438   0.070641  ...   0.175230 -0.064081 -0.143474 -0.161579
Medu     -0.163658  1.000000  0.623455  -0.171639  ...   0.100285  0.205341  0.215527  0.217147
Fedu     -0.163438  0.623455  1.000000  -0.158194  ...   0.024473  0.190270  0.164893  0.152457
traveltime 0.070641 -0.171639 -0.158194   1.000000  ...  -0.012944 -0.093040 -0.153198 -0.117142
studytime -0.004140  0.064944 -0.009175  -0.100909  ...  -0.062700  0.160612  0.135880  0.097820
failures   0.243665 -0.236680 -0.250408   0.092239  ...   0.063726 -0.354718 -0.355896 -0.360415
famrel     0.053940 -0.003914 -0.001370  -0.016808  ...  -0.044354  0.022168 -0.018281  0.051363
freetime   0.016434  0.030891 -0.012846  -0.017025  ...  -0.058078  0.012613 -0.013777  0.011307
goout      0.126964  0.064094  0.043105   0.028540  ...   0.044302 -0.149104 -0.162250 -0.132791
Dalc       0.131125  0.019834  0.002386   0.138325  ...   0.111908 -0.094159 -0.064120 -0.054660
Walc       0.117276  0.047123  0.012631   0.134116  ...   0.136291 -0.126179 -0.084927 -0.051939
health     -0.062187 -0.046878  0.014742   0.007501  ...  -0.029937 -0.073172 -0.097720 -0.061335
absences   0.175230  0.100285  0.024473  -0.012944  ...   1.000000 -0.031003 -0.031777  0.034247
G1         -0.064081  0.205341  0.190270  -0.093040  ...  -0.031003  1.000000  0.852118  0.801468
G2         -0.143474  0.215527  0.164893  -0.153198  ...  -0.031777  0.852118  1.000000  0.904868
G3         -0.161579  0.217147  0.152457  -0.117142  ...   0.034247  0.801468  0.904868  1.000000
```

10.

```
Possible Target Variables:  
['G1', 'G2', 'G3']
```

PART B – TEXT DATA

Using the feedback dataset:

1. Read the text file.
2. Count the number of sentences.
3. Count the number of words.
4. Find the 10 most frequently occurring words.
5. Remove common stop words and compare the results.

ADDITIONAL TEXT ANALYTICS TASK

6. Calculate vocabulary size.
7. Find average sentence length.
8. Generate a word cloud.

CODE:

```
import pandas as pd  
  
import nltk  
  
from nltk.corpus import stopwords  
  
from nltk.tokenize import sent_tokenize, word_tokenize  
  
from collections import Counter  
  
from wordcloud import WordCloud  
  
import matplotlib.pyplot as plt  
  
  
# 1. Read the dataset  
  
df = pd.read_csv(r"D:\SEM_5\ML\datasets\Text_data\IMDB Dataset.csv")  
  
reviews = df['review']  
  
text = " ".join(reviews.astype(str))  
  
  
# 2. Count the number of sentences  
  
sentences = sent_tokenize(text)  
  
print("\nNumber of Sentences:")  
  
print(len(sentences))
```

3. Count the number of words

```
words = word_tokenize(text)
```

```
words = [word.lower() for word in words if word.isalpha()]
```

```
print("\nNumber of Words:")
```

```
print(len(words))
```

4. Find the 10 most frequent words

```
word_freq = Counter(words)
```

```
print("\nTop 10 Most Frequent Words:")
```

```
for word, count in word_freq.most_common(10):
```

```
    print(f"{word} : {count}")
```

5. Remove stop words and compare results

```
stop_words = set(stopwords.words('english'))
```

```
filtered_words = [word for word in words
```

```
                    if word not in stop_words]
```

```
filtered_freq = Counter(filtered_words)
```

```
print("\nTop 10 Words After Removing Stop Words:")
```

```
for word, count in filtered_freq.most_common(10):
```

```
    print(f"{word} : {count}")
```

6. Calculate Vocabulary Size

```
vocabulary = set(filtered_words)
```

```
print("\nVocabulary Size:")
```

```
print(len(vocabulary))
```

7. Find Average Sentence Length

```
total_words = len(words)
```

```
total_sentences = len(sentences)
```

```
avg_sentence_length = total_words / total_sentences
```

```
print("\nAverage Sentence Length:")
```

```
print(round(avg_sentence_length, 2), "words")
```

8. Generate Word Cloud

```
wordcloud = WordCloud(  
    width=1000,  
    height=500,  
    background_color='white'  
).generate(" ".join(filtered_words))  
plt.figure(figsize=(12, 6))  
plt.imshow(wordcloud, interpolation='bilinear')  
plt.axis("off")  
plt.title("Word Cloud of IMDb Reviews")  
plt.show()
```

OUTPUT:

2.

```
Number of Sentences:  
528298
```

3.

```
Number of Words:  
11372037
```

4.

```
Top 10 Most Frequent Words:  
the : 664213  
and : 323076  
a : 321274  
of : 288526  
to : 267033  
is : 216900  
br : 201951  
it : 187987  
in : 185215  
i : 173731
```



```
Top 10 Words After Removing Stop Words:
br : 201951
movie : 85070
film : 76919
one : 51828
like : 39183
good : 28767
would : 26475
even : 24780
time : 23481
really : 23024
```

Vocabulary Size:
96882

Average Sentence Length:
21.53 words

[illegible]

PART C – IMAGE DATA

Using the image dataset:

1. Load an image.
2. Display the image.
3. Determine image dimensions.
4. Determine whether it is grayscale or RGB. Perform conversions
5. Calculate the total number of pixels.

CODE:

```
import pickle
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
def unpickle(file):
```

```
    with open(file, 'rb') as fo:
```

```
        data = pickle.load(fo, encoding='bytes')
```

```
    return data
```

```
# 1. Load an Image
```

```
batch = unpickle(r"D:\SEM_5\ML\datasets\Image_data\cifar-10-batches-py\data_batch_1")
```

```
image = batch[b'data'][0]
```

```
image = image.reshape(3, 32, 32)
```

```
image = np.transpose(image, (1, 2, 0))
```

```
# 2. Display the Image
```

```
plt.imshow(image)
```

```
plt.title("CIFAR-10 Image")
```

```
plt.axis('off')
```

```
plt.show()
```

```
# 3. Determine Image Dimensions
```

```
height, width, channels = image.shape
```

```
print("Image Dimensions:")
```

```
print("Height =", height)
```

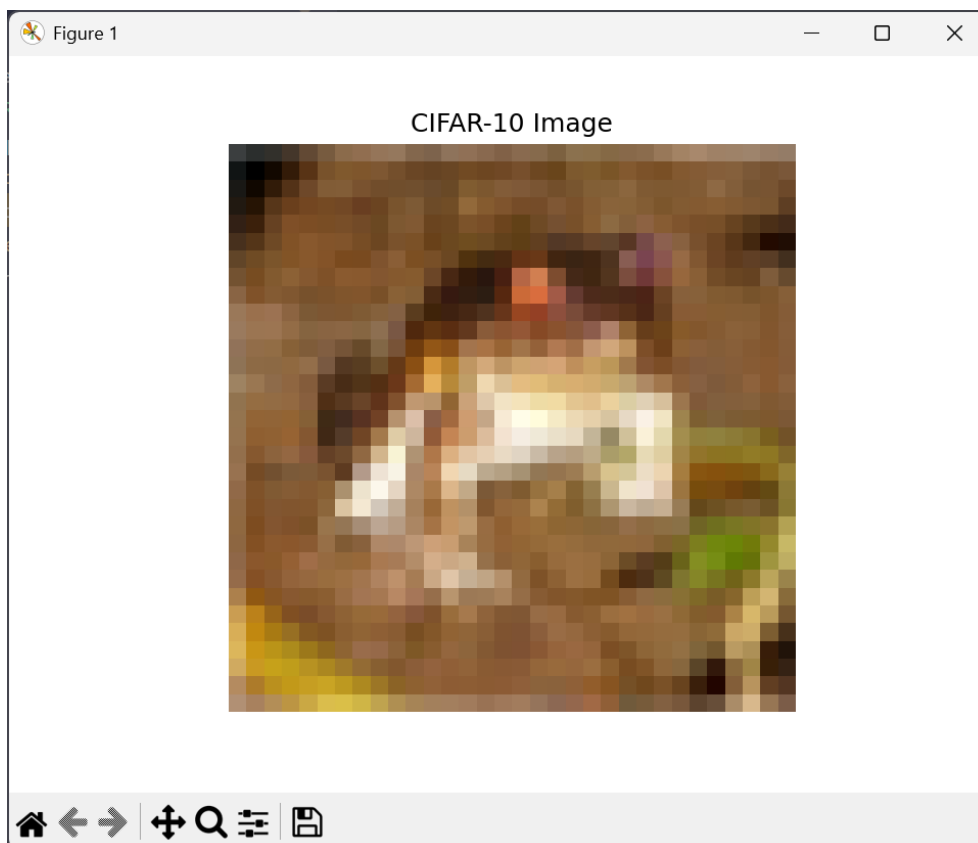
```
print("Width =", width)
print("Channels =", channels)

# 4. RGB or Grayscale
if channels == 3:
    print("\nImage Type: RGB")
    gray = np.mean(image, axis=2)
    plt.imshow(gray, cmap='gray')
    plt.title("Grayscale Image")
    plt.axis('off')
    plt.show()
else:
    print("\nImage Type: Grayscale")

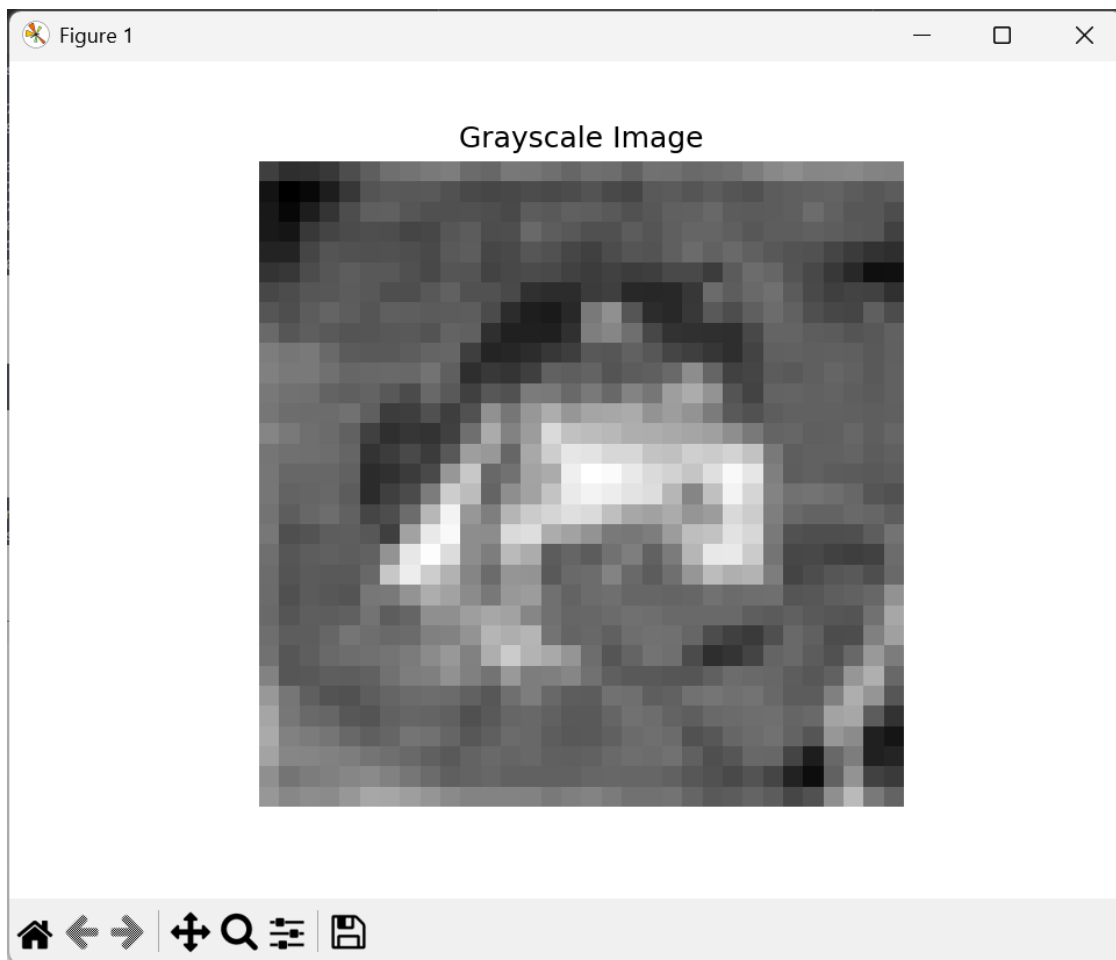
# 5. Total Number of Pixels
pixels = height * width
print("\nTotal Number of Pixels =", pixels)
```

OUTPUT:

1.



2.



3.

```
Image Dimensions:  
Height = 32  
Width = 32  
Channels = 3
```

4.

```
Image Type: RGB
```

5.

```
Total Number of Pixels = 1024
```